

QC Automation Plan — Project NITRO

How QC will be automated using Claude Code (restricted) — without Claude Cowork

Author	Thong Ho (QC)
Prepared for	Joel Dawson
Project	Project NITRO (Jira: NIT, Confluence: PN1)
Date	2026-07-10
Status	Draft for review

1. Objective

Automate the QC process for Project NITRO end-to-end using Claude Code within the approved security sandbox — without Claude Cowork. The pipeline generates test cases from the Functional Specs, builds and runs E2E automation, captures failures as evidence, logs defects to Jira automatically, and reports coverage against release exit criteria.

2. Current status

- Automation framework built and validated: Playwright + TypeScript, Page Object Model, auto bug → Jira reporter, QC report generator.
- Full workflow proven end-to-end to validate the pipeline (demo video + bug report shared in the channel).
- Claude Cowork was not approved. Claude Code access is being provisioned (pending). Claude Desktop will come later.

3. Access I have

- Source repository (Hub24-Automation) and local dev environment (VS Code).
- The Playwright automation framework (pages, tests, reporters, CI config) already in the repo.
- Jira (NITRO / NIT) and Confluence (space PN1) via my browser account.
- Jira REST API access via an API token stored in .env (used by the auto bug → Jira reporter).
- CI: CloudBees pipeline config present in the repo.

4. Access I need

- **Claude Code provisioned** for my account (currently blocked / pending).
- **Network allowlist additions** in the Claude Code settings.json so tests and the Jira integration can run inside the sandbox (see Section 6). This is the top blocker.
- **A Jira account with create-issue permission on NITRO** (a HUB24 account) + a matching API token for the automation.
- **Test environment access + mock/test data** for the real HUB24 app (to run against the real application).
- **CI runner egress** to the same domains so the suite runs in CloudBees, not just locally.

5. Automation workflow (how I will use Claude Code)

1. Generate test cases from the Functional Specs (Confluence) — publish to Confluence and save `.feature/.json` in the repo.
2. Write and maintain the E2E automation (Playwright, Page Object Model).
3. Run the E2E suite locally and in CI (CloudBees).
4. Capture each failure as evidence: bug-report (ticket description), bug-image (screenshot), bug-video.
5. Auto-create / update Jira defects: Epic parent → Bug, Story parent → Bug Fixing (in-sprint) Sub-task; de-duplicate existing tickets; reopen on re-fail, close on pass.
6. Produce a QC report: coverage, pass rate, and open defects against release exit criteria.

6. Domains to whitelist

The current `settings.json` uses `allowManagedDomainsOnly = true`, so anything not listed is blocked. The following need to be added to `network.allowedDomains` for the workflow to function inside the sandbox:

Domain	Purpose	Priority
<code>api.atlassian.com</code>	Jira REST API — auto-create / update defect tickets	High
<code>classltd.atlassian.net</code>	Jira (NITRO) + Confluence (PN1) site	High
<code>*.atlassian.net</code>	Atlassian site assets / redirects	High
<HUB24 test environment URL>	The app under test (E2E target) — please confirm the UAT/test domain	High
<code>cdn.playwright.dev</code>	Download Playwright browser binaries (npx playwright install)	High
<code>playwright.download.prss.microsoft.com</code>	Playwright browser binary CDN (mirror)	Med
<code>*.browserstack.com</code>	Cross-browser runs (only if BrowserStack is used)	Optional
<code>*.chromatic.com</code>	Visual regression uploads (only if Chromatic is used)	Optional

Note: `registry.npmjs.org` and `github.com` are already allowed, so `npm` installs and `git` operations work.

7. Working within the security restrictions

Restriction (<code>settings.json</code>)	How the workflow complies / impact
Sandbox on; only allow-listed commands	Our stack only needs <code>npm</code> / <code>npx</code> / <code>git</code> / <code>tsc</code> — all allow-listed. No impact.
<code>.env</code> and secrets not readable by the agent	Tests load <code>.env</code> at runtime; the agent never reads secrets. Good for security — no impact on running tests.
No Jira / Slack MCP (managed MCP only)	Defect creation uses the Jira REST API (token in <code>.env</code>), not an MCP — needs the domains in Section 6.

Restriction (settings.json)	How the workflow complies / impact
No WebFetch / network allow-list only	All required endpoints must be whitelisted (Section 6).
All actions audited / telemetry on	Acceptable — full traceability of every command and edit.

8. Risks & dependencies

- **Network allow-list** missing the test-target and Atlassian domains → blocks both test execution and Jira push. (Top priority.)
- **Test environment + mock data** pending → blocks running against the real HUB24 app.
- **Claude Code access** pending provisioning.
- **Jira workflow transition names** must match what the reporter uses (configurable) for auto status changes.

9. Requests (to proceed)

7. Approve the domain allow-list additions in Section 6.
8. Confirm the HUB24 Jira account + create-issue permission on NITRO.
9. Provision Claude Code for my account.
10. Provide test environment access and mock/test data for the HUB24 app.